ETCIM: Error-Tolerant Digital CIM Processor With Redundancy-Free Hard Error Repair and Run-Time Soft Error Correction

Yiqi Wang, *Graduate Student Member, IEEE*, Zhen He, *Graduate Student Member, IEEE*, Chenggang Zhao, Zihan Wu, Mingyu Gao, Huiming Han, Shaojun Wei, *Fellow, IEEE*, Yang Hu, *Member, IEEE*, Fengbin Tu, *Member, IEEE*, and Shouyi Yin, *Fellow, IEEE*

***Abstract*— Digital computing-in-memory (CIM) has demonstrated outstanding energy and area efficiency. Although digital CIM avoids errors along the in-memory computing path, it is indispensable to correct hard and soft errors in digital CIM's SRAM cells like conventional SRAM for commercial products. Hard errors come from defects in chip fabrication. Soft errors result from bit flips in an SRAM cell during CIM computation, and cause multiply–accumulation (MAC) errors. However, traditional SRAM's error correction scheme cannot be directly applied to digital CIM, which causes hardware overhead in hard error repair, and incompatibility and latency problems in soft error correction. To tackle the above challenges, we propose an error-tolerant digital CIM processor ETCIM with three corresponding features: 1) a fault-adaptive CIM initializer (FACI) and input feeder reduce the hard error repair overhead by up to 8.47× power and 18.16× area savings; 2) a blockwise error correction code (BW-ECC) CIM macro suits the digital CIM structure that integrates multiple weight blocks, and corrects soft MAC errors; and 3) a progressive cell error corrector (PCEC) hides soft cell error correction latency during computation, reducing the total latency by up to 96.8%. The fabricated ETCIM achieves a state-of-the-art energy efficiency with an error correction of 19.5 tera operations per second (TOPS)/W at INT8.**

***Index Terms*— Computing-in-memory (CIM), error correction code (ECC), error, error-tolerant design, fault.**

Received 13 December 2024; revised 30 March 2025; accepted 24 April 2025. Date of publication 20 May 2025; date of current version 30 January 2026. This article was approved by Associate Editor Sanu Mathew. This work was supported in part by the National Key Research and Development Program under Grant 2023YFB4403100; in part by NSFC under Grant 62422407, Grant 62125403, Grant 92464302, Grant U24B20164, and Grant 92164301; in part by Shanghai Municipal Science and Technology Major Project; in part by the Natural Science Foundation of Jiangsu Province Basic Research Program under Grant BK20243042; in part by Beijing National Research Center for Information Science and Technology; and in part by Beijing Advanced Innovation Center for Integrated Circuits. *(Corresponding authors: Fengbin Tu; Shouyi Yin.)*

Yiqi Wang, Zhen He, Zihan Wu, Huiming Han, Shaojun Wei, Yang Hu, and Shouyi Yin are with the School of Integrated Circuits, Beijing Innovation Center for Future Chip, and Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: yinsy@tsinghua.edu.cn).

Chenggang Zhao and Mingyu Gao are with the Institute for Interdisciplinary Information Sciences, Tsinghua University, Beijing 100084, China.

Fengbin Tu is with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, China (e-mail: fengbintu@ust.hk).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSSC.2025.3566283.

Digital Object Identifier 10.1109/JSSC.2025.3566283

I. Introduction

DUE to the increasing size and computation of neural network (NN) models, conventional digital artificial intelligence (AI) processors usually suffer from massive data movements between separate multiply–accumulation (MAC) units and memory. Computing-in-memory (CIM) overcomes the "memory wall" problem by integrating computation into memory. Among diverse CIM techniques, digital CIM is currently one of the most promising architectures [12]. Unlike conventional analog CIM, digital CIM integrates customized digital MAC logic into SRAM, enabling multi-row parallel computation [29]. Digital CIM has demonstrated outstanding area and energy efficiency [12], [24], [25], [26], [27], [32]. Besides, its power–performance–area (PPA) scales well with technology scaling [6], [11], [12], [18], avoiding the poor scaling of analog circuits.

However, although digital CIM employs adders to accumulate multiplication results instead of analog additions through current or voltage like analog CIM, it only avoids the errors from multi-row addition. Digital CIM's SRAM cells are still prone to hard and soft errors as in conventional SRAM. Fig. 1 depicts the hard and soft error mechanisms in digital CIM. Hard errors come from defects in chip fabrication. The mismatch between the mapped weight bit and the corresponding SRAM cell fault type causes a hard error after weights are mapped to CIM. Soft errors result from bit flips in an SRAM cell during CIM computation and cause MAC errors. Therefore, how to correct errors in digital CIM's SRAM cells to boost digital CIM's reliability and robustness is the key for digital CIM's real deployment in commercial products.

Although traditional SRAM has mature error correction schemes such as column repair for hard errors and hamming error correction code (ECC) code for soft errors, they cannot be directly applied to digital CIM, which causes hardware overhead in hard error repair, and incompatibility and latency problems in soft error correction (see Fig. 2).

1) *Challenge1 (Hard Error):* Traditional SRAM uses column repair to select out correct bits from a read-out row. However, digital CIM reads out multiple rows simultaneously for computation. Applying traditional column repair to digital CIM incurs massive detour

0018-9200 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

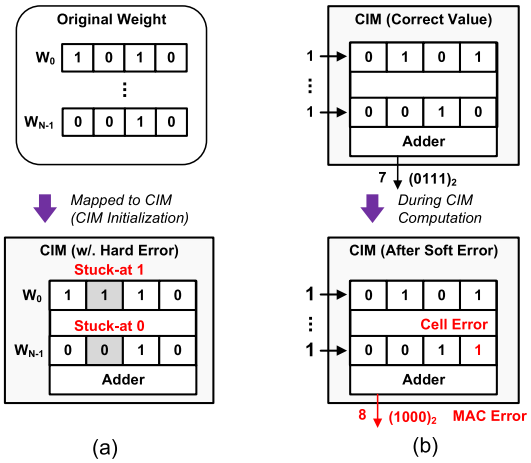


Fig. 1. (a) Hard and (b) soft error mechanisms in digital CIM.

overhead for each read-out row, causing 24.3% area and 14.3% power overhead.

2) *Challenge2a (Soft MAC Error):* Traditional ECC schemes for SRAM [14] can only correct single-bit errors in a row. During digital CIM's row addition, an SRAM cell flip may cause multiple bit flips in the multi-bit MAC result, and the traditional ECC is incompatible.
3) *Challenge2b (Soft Cell Error):* After MAC error correction, digital CIM still needs to correct the original flipped SRAM cell that causes MAC errors, so as to avoid uncorrected cell errors' accumulation. However, this incurs high latency in stalling computation to read out SRAM rows in turn.

This article proposes the first error-tolerant digital CIM processor ETCIM. It features comprehensive error tolerance techniques tailored for digital CIM, including redundancy-free hard error repair and run-time soft MAC and cell error correction. ETCIM overcomes the above challenges of error-tolerant designs in digital CIM with the following three features.

1) For *Challenge1*, we use a fault-adaptive CIM initializer (FACI) and input feeder, which avoids detour and redundancy logic and reduces hard error repair overhead to 1.9% area and 2.0% power. FACI adaptively flips weights to match faulty SRAM cells' distribution, and the input feeder inverts inputs accordingly to ensure computation equivalence.
2) For *Challenge2a*, we design a blockwise ECC (BW-ECC) CIM macro to correct soft MAC errors. When a soft error happens, rather than protecting flips of individual bits like traditional ECC, BW-ECC directly adds the cell error-induced deviation back into the faulty block's victim MAC result for error correction. The faulty block is located by the residue code, and the MAC result deviation is indicated by the checksum difference.
3) For *Challenge2b*, we exploit a progressive cell error corrector (PCEC) to hide the soft cell error correction latency during computation without stall. Once an MAC error is detected, in subsequent computation, PCEC utilizes each computation cycle's MAC correctness to check if the current cycle's activated rows contain the error row, and gradually reduces the potential error row (PER) scope until localization. Since different error columns in the faulty block can be distinguished via their unique checksum difference, the cell error can be corrected with the known error row and column.

This article is the journal extension version of our VLSI'24 work [31], providing higher level insights to readers with many additional materials. The rest of this article is organized as follows. Section II provides background and motivation. Section III proposes ETCIM's overall architecture. The three architectural features are described in Sections IV–VI, respectively. Section VII presents our experimental results on the 28-nm ETCIM processor. Section VIII concludes this article.

II. Background and Motivation

A. Digital CIM

As shown in Fig. 3(a) and (b), instead of using analog-to-digital converters (ADCs) and digital-to-analog converters (DACs) to perform analog MAC like conventional analog CIM [22], [30], [34], the digital CIM works usually customize each SRAM cell with an additional two-input NOR for bitwise multiplication and place digital adders near the SRAM array for partial sum accumulation. Their differences lie in their realization approach of accumulating multiplication results: analog accumulation or digital accumulation. Although digital CIM avoids the errors along the in-memory accumulation path using digital adders, its SRAM cells are still prone to hard and soft errors, similar to conventional analog CIM and SRAM.

B. Errors in Digital CIM

Hard errors mainly come from manufacturing defects like undesired chemical and airborne particles deposited on the chip [16], which cause actual damage to the CIM. The most common hard errors are stuck-at faults, where an SRAM cell is permanently stuck at a "0" or "1" and cannot be overwritten with the complementary data value. *Soft errors* occur when an unexpected factor like voltage fluctuations interrupts the data read [8] during CIM computation. They are temporary in that once a new value is written to the affected word, the error no longer exists.

Specifically, with technology scaling, both hard and soft errors become increasingly widespread. *For hard errors*, advanced nodes pack billions of transistors into a similar or smaller chip area, resulting in much higher integration density where the SRAM cells are closer together. Therefore, a single defect can, thus, affect a larger number of cells, causing hard errors to occur more frequently. Besides, the high integration density necessitates the cells to shrink to extremely tiny dimensions, making them highly sensitive to manufacturing variations like a particle or misalignment. The transistor shrinking also makes maintaining SRAM stability more difficult. Cells that do not meet stability requirements are considered defective, lowering the yield and networks' inference accuracy. Taking the 100-mm^2 5-nm die as an example, which is roughly the size of the first batch of mobile processors coming out of TSMC's process, the manufacturing yield is only 32.0% without self-repair circuitry [2]. *For soft errors*, the operating voltage (Vop) of SRAM transistors reduces as the technology node advances, while the threshold voltage

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

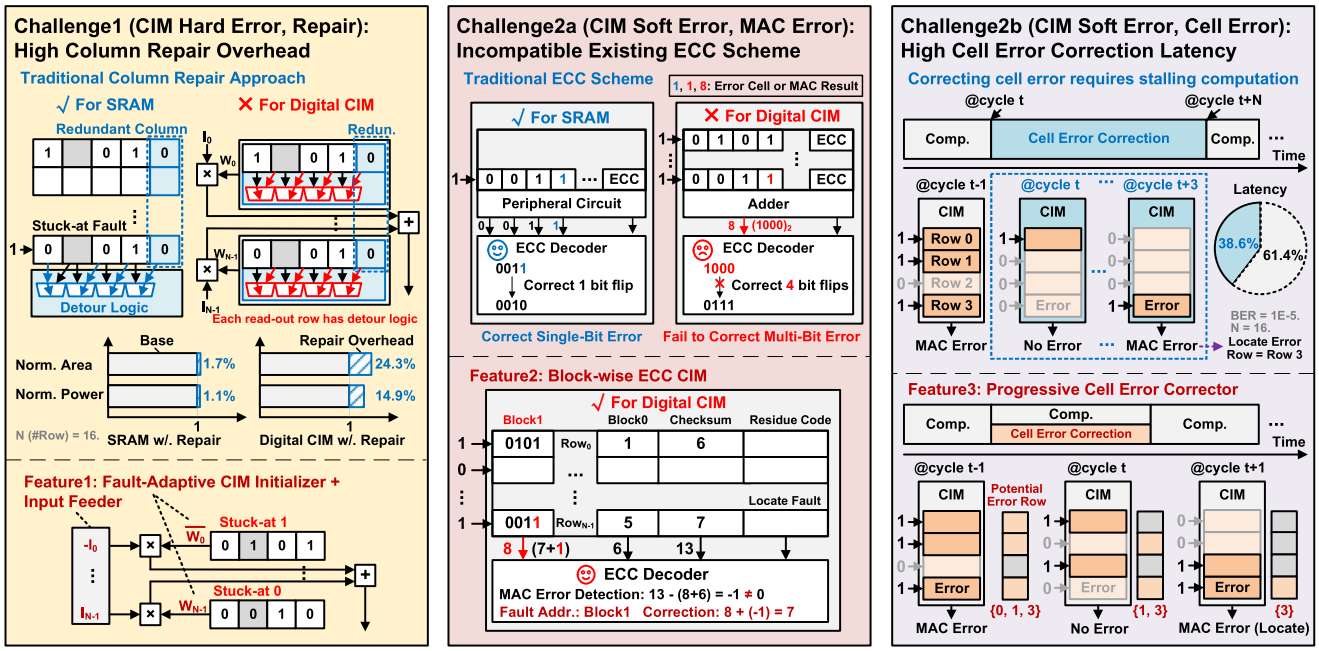


Fig. 2. Digital CIM's unique structure and error mechanism raise new challenges for error-tolerant design. ETCIM has corresponding features that target the challenges: FACI and input feeder that repair hard errors, BW-ECC CIM that corrects soft MAC errors, and PCEC that hides soft cell error correction latency.

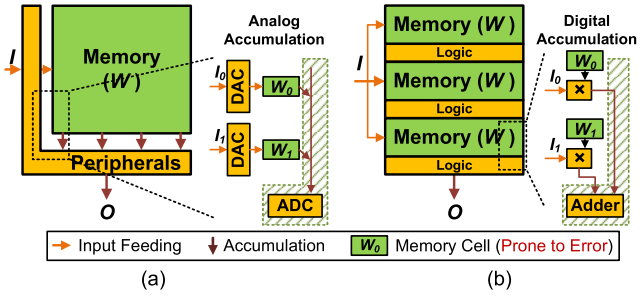


Fig. 3. CIM processor architectures. (a) Analog CIM. (b) Digital CIM.

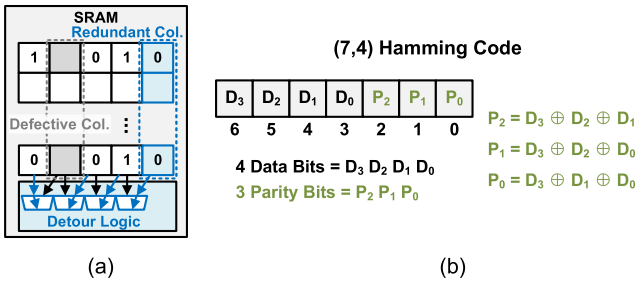


Fig. 4. Existing error correction techniques for SRAM. (a) Column repair for hard errors. (b) Hamming code-based ECC for soft errors.

(Vth) reduces slower due to gate engineering limitations [21]. This decreases voltage headroom (Vop–Vth), reducing noise immunity and increasing soft error possibilities. Therefore, correcting hard and soft errors becomes more critical at advanced technology nodes.

C. Existing Error Correction Techniques for SRAM

Traditional SRAM has mature schemes to repair hard errors and correct soft errors, as depicted in Fig. 4. *For hard errors*, redundant columns are often added to the primary SRAM array as spares so that the defective column can be replaced with the redundant column as needed. The defective column replacement is realized by the detour logic in the read-out peripheral, which bypasses the faulty SRAM cells in the defective column and selects out correct bits from other columns in the primary SRAM array and the redundant column. Besides the redundancy repair, assist circuits can also be used to address hard errors in SRAM design [13], [20]. Assist circuits are typically integrated into each cell or the array periphery, involving additional hardware such as capacitors, charge pumps, or level shifters to improve the stability of SRAM cells during read and write operations. Their differences mainly lie in the cost and usage frequency. Compared to redundancy repair, assist circuits typically incur higher overhead because they may require additional transistors or control logic for each cell or column, and need to remain active continuously. Besides, redundancy repair is a standardized practice for improving yield in SRAM design, while assist circuits are more suited to specific needs, such as low-voltage operation or high stability requirements. To sum up, despite the presence of assist circuits, redundancy repair is still indispensable and more commonly used.

For soft errors, hamming code-based ECC [14] is typically exploited to provide single-bit error correction and double-bit error detection (SECDED), aiming to protect single bit flip in one read-out row. In hamming code encoding, data bits are selectively XORed to form parity bits. In decoding, the read-out data bits are XORed using the same method as encoding and compared with the read-out parity bits. If some comparisons are mismatched, the faulty bit address can be decoded according to the mismatched parity bit positions. Since multi-bit errors in a row are rare in regular environments [17], [23], typical SECDED ECC can meet most error-resilience cases,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

which is widely used in industry [1], [15], [19] such as AMD [1] and NVIDIA [19]. Thus, we also only consider correcting single bit flip on chip.

D. Digital CIM's Error Correction Requirements

However, digital CIM's unique structure raises new error correction requirements, rendering conventional error correction techniques for SRAM unsuitable for digital CIM. The main structural difference between digital CIM and SRAM is that digital CIM reads out multiple rows simultaneously to calculate MAC results, while SRAM reads out only one row in each cycle. This brings digital CIM's unique error mechanism, whose correction difficulty is much higher than that of SRAM. *For hard error correction*, any read-out row may have defective SRAM cells, so the expensive detour logic is inevitable for each row if column repair is applied. *For soft error correction*, both MAC and cell error correction are indispensable. MAC error correction maintains computation accuracy, and cell error correction eliminates the root cause of MAC errors to avoid uncorrected cell errors' accumulation. If we only correct MAC errors without cell error correction, when another cell error occurs, the MAC error cannot be corrected as accumulated cell errors exceed the ECC capability. However, the traditional hamming code-based ECC fails to meet the protection needs of both computation and storage in digital CIM. The MAC results may occur multiple bit flips due to the flipped SRAM cell-induced carry propagation (see Fig. 1), which exceeds the single error correction capability of the traditional ECC. Besides, digital CIM's MAC results merge the information of multiple read-out rows and confuse the distinction between correct and faulty SRAM cells. Therefore, digital CIM needs to correct cell errors in any possible read-out row after MAC error correction, necessitating time-consuming row-by-row reading out to locate the error row.

E. Motivation of ETCIM

In this article, we will discuss how to leverage the computational and storage characteristics of digital CIM to overcome the above error correction challenges. *For hard errors*, we notice that the stored weights do not need to be exactly the same as the original weights. Digital CIM only needs to maintain computational correctness, so we can adaptively flip weights to match faulty SRAM cells and invert inputs accordingly, achieving redundancy-free hard error repair. *For soft MAC errors*, since multiple weights are concatenated and stored in a CIM row, digital CIM can be divided into multiple weight blocks according to the weight precision and each block outputs an MAC result in each cycle. Although we can employ checksum to obtain the deviation between the victim MAC result and the correct result in the faulty block, how to locate the faulty block among multiple weight blocks is quite challenging. Traditional ECC uses bitwise XOR to protect individual data bits, but digital CIM needs to protect many multi-bit weights, and XOR-based protection fails to maintain effectiveness during multi-row addition [9]. In this work, we propose BW-ECC that suits digital CIM's storage characteristics. It combines hamming code and residue code to maintain ECC's effectiveness during addition, enabling the faulty block localization based on weights' and ECC's MAC results. *For soft cell errors*, we take the opportunity that each computation cycle's MAC correctness can be used to indicate if the error row is activated. Besides, since MAC errors can be corrected by BW-ECC in real time, digital CIM can maintain computational correctness in the presence of cell error. Thus, we hide the soft cell error correction latency during computation and utilize each computation cycle's MAC correctness and different activated rows to gradually locate and correct the error row.

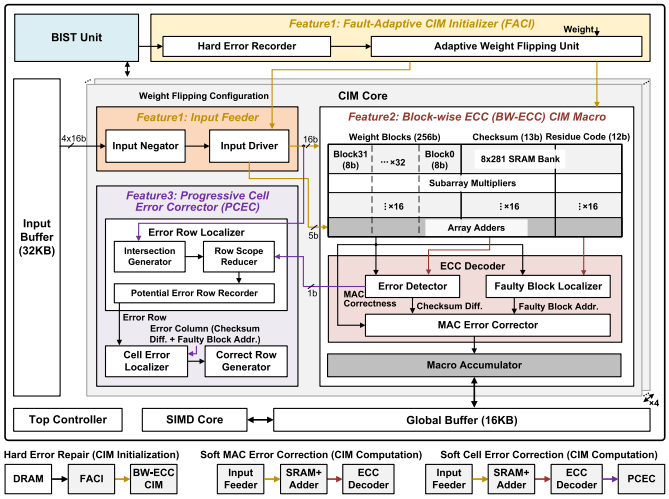


Fig. 5. ETCIM processor's overall architecture, with three configurations for hard error repair, soft MAC error correction, and soft cell error correction.

To sum up, ETCIM employs different error correction approaches for hard and soft errors due to their different error mechanisms. For hard errors, we do not correct the original faulty SRAM cell, as it comes from manufacturing defects and cannot be overwritten with the correct value. On the contrary, we counteract the hard errors' impacts, which maintains digital CIM's functional correctness with the presence of hard errors. For soft errors, since the flipped SRAM cell can be overwritten with the correct value, we locate and correct the original soft cell error directly, which eliminates the root cause of soft errors and solves the problem fundamentally. Therefore, both hard and soft error corrections enable digital CIM to operate correctly under a certain rate of hard and soft errors, which improves digital CIM's error tolerance capability. Sections III–VI will go deeper into the implementation details of these error tolerance techniques.

III. ETCIM Processor Architecture

The ETCIM processor's overall architecture is shown in Fig. 5, with three features highlighted in different colors. It consists of four CIM cores, an FACI, a built-in self-test (BIST) unit, a 32-kB input buffer, a 16-kB global buffer, a single instruction multiple data (SIMD) core, and a top controller. The CIM core includes an input feeder, a PCEC, and a BW-ECC CIM macro, whose SRAM cells are constructed using the logic design rule. Each macro comprises 16 8 $\times$ 281 subarrays, and each CIM row has 256-b weight data, 13-b checksum, and 12-b residue code. The minimal support weight precision is 8 b, so the 256-b weight data of the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

CIM macro can be divided into 32 weight blocks according to the weight precision. Each weight block corresponds to different output channels, so there are 32 output channels per row. The total stored weight data of all subarrays are "128 input channels × 32 output channels," which follows the prior state-of-the-art digital CIMs' weight mapping scheme [6], [12], [18]. In each cycle, each subarray activates one row, and each block accumulates the multiplication results from 16 rows and generates an MAC result in parallel. Therefore, the total read-out weight data are "16 input channels × 32 output channels." The eight rows of each subarray are activated sequentially. Since different weights across the 128 rows of the same weight block contribute to the same result in one output channel, the access order of the rows within subarrays does not affect the correctness of partial sum accumulation.

The reason why each block corresponds to different output channels is that the CIM macro has the input broadcasting characteristic, where input is shared across all weights in a row. Since inputs are multiplied with multiple weights in different output channels in NNs, storing different output channels' weights in different blocks achieves a match between the algorithm property and hardware characteristic. If each block corresponds to different input channels, 32 inputs need to be unicast to 32 8-b weights in each row, resulting in expensive wiring overhead.

ETCIM has different configurations for hard error repair, soft MAC error correction, and soft cell error correction. The different modes reuse the same datapath to save hardware resources. In CIM initialization, FACI performs adaptive weight flipping for each weight row and maps them onto CIM macros to repair hard errors. In CIM computation, the input feeder inverts inputs sent to flipped weights for MAC operations. If the checksum difference is not zero, the soft error is detected by the error detector. Then, the faulty block localizer exploits the residue code to locate the faulty block, and the MAC error corrector restores the correct MAC result by adding the checksum difference to the victim MAC result. The macro accumulator completes partial sum accumulation for correct MAC results, and CIM outputs are written to the global buffer with activation functions performed by the SIMD core. Meanwhile, PCEC is activated to reduce the PER scope until localization. After the error row is found, the cell error position can be located since the error column can be inferred by the checksum difference and faulty block address. Finally, the correct row is generated and rewritten back to CIM to correct the cell error.

IV. FACI and Input Feeder

A. Hard Error Repair Workflow

Fig. 6 uses an example to explain the hard error repair workflow. The original weights W_0 and W_1 are $(1010)_2$ and $(0010)_2$, respectively, but the second column of the SRAM to be mapped is defective, which has a stuck-at 1 fault and stuck-at 0 fault on the two rows. If we map the two weights onto the CIM macro directly without column repair, the mismatch between the mapped weight bit and the corresponding SRAM cell fault type will cause a hard error in the first row.

To avoid expensive detour logic incurred by column repair, in this work, we adaptively flip weights to match faulty SRAM

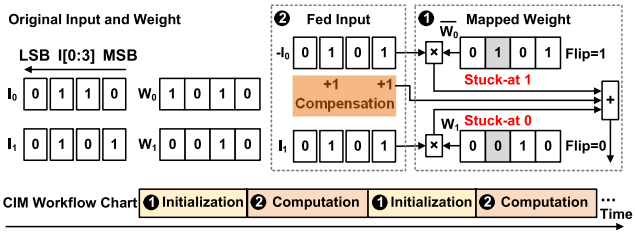


Fig. 6. Hard error repair workflow example. ❶ Weights are adaptively flipped to match faulty SRAM cells' distribution in CIM initialization. ❷ Inputs are inverted accordingly in CIM computation.

cells' distribution, instead of bypassing faulty SRAM cells to match weight values. If the original weight W cannot be stored correctly in CIM, the $\overline{W}$ can always be, ensuring that either the correct W or $\overline{W}$ is stored successfully. In this example, W_0's second bit (0) mismatches the corresponding stuck-at 1 fault but W_1's second bit (0) matches the corresponding stuck-at 0 fault. Therefore, we flip W_0 to form $\overline{W_0}$ and remain W_1 unchanged, and map them onto CIM. Meanwhile, we record that the flip signal of the first row equals 1, which guides the corresponding input inversion.

Note that to avoid stuck-at faults at any possible SRAM cell position, we need to flip all weight bits and map $\overline{W}$ rather than $-W$ if a mismatch happens. This is because due to the 2's complement representation, $-W$ equals $\overline{W}+1$, and a few LSB bits in $-W$ are the same with W. If these bits of W mismatch the corresponding SRAM cell fault type, the mismatch still exists when mapping $-W$. In other words, mapping neither $-W$ nor W can repair the hard error in some cases, as both of their mapping results are not the correct value. Therefore, we choose to store W or $\overline{W}$ to ensure that at least one of them can be stored correctly. When the $\overline{W}$ is stored, an additional 1 should be added to $\overline{W}$ to transform it to $-W$ if the current input bit is not zero. The $+1$ compensation is added directly to the final result, and its implementation details will be elaborated in Section IV-B.

Besides addressing a single hard error in a weight, our redundancy-free repair can also handle the scenario where two hard errors occur in a weight in certain situations. In fact, our adaptive weight flipping has the ability to counteract the impacts of at least one hard error per row. Even if two hard errors occur in a weight, whose probability of occurrence is quite low, we can still maintain the computational correctness as long as the two SRAM cell fault types are all mismatched or not mismatched with the corresponding mapped weight bits. If they are all matched, we map the original weight directly to CIM. Otherwise, we flip the weight and map since the flipped weight can be successfully stored in CIM.

B. FACI and Input Feeder Architecture

After discussing the hard error repair workflow, we go deeper into FACI and input feeder's detailed architecture to explain how the proposed workflow is implemented in hardware, as illustrated in Fig. 7. FACI comprises a hard error recorder and an adaptive weight-flipping unit. The BIST unit detects the list of hard errors from fabrication after the chip is powered on, and stores it in FACI. For each SRAM row to be mapped, FACI first checks the hard error list. If this row has a stuck-at fault and the cell fault type mismatches the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

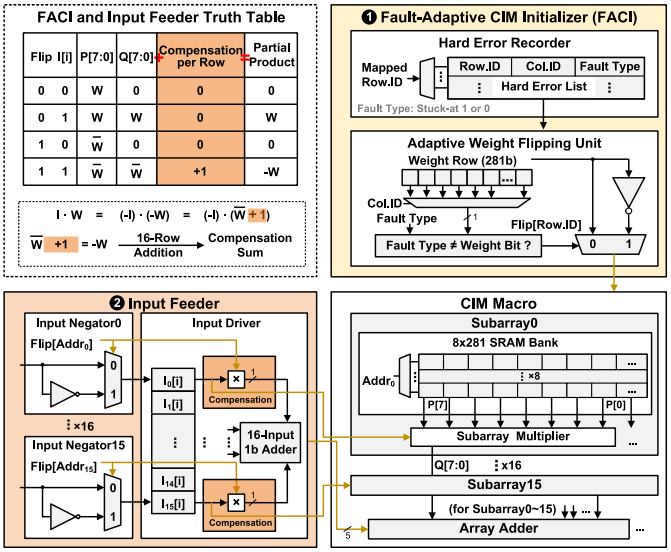


Fig. 7. FACI and input feeder architecture.

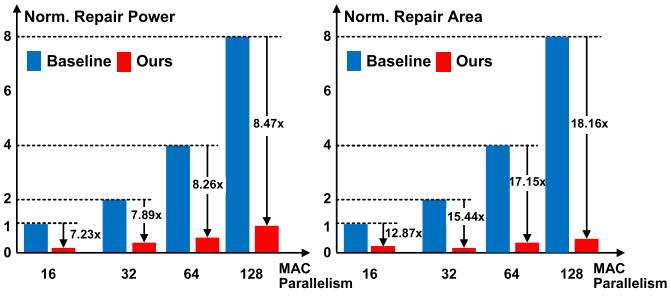


Fig. 8. Evaluation for the FACI and input feeder on hard error repair overhead reduction. Baseline: conventional column repair method, where each read-out row in digital CIM has a redundant column and detour logic for repair. MAC parallelism is the count of weight rows that are simultaneously read out and accumulated to complete MAC operations.

corresponding weight bit, FACI flips all bits of the weight row and maps them onto CIM. Since the CIM macro has 16 subarrays and each subarray reads out one row in each cycle, we equip 16 input negators in the input driver. Each of them controls whether to invert the input based on the flip signal of the activated weight row. After input inversion, the input driver generates the compensation for each read-out row. If the weight row just stores the original W, the compensation is zero. Otherwise, a $+1$ compensation is required to transform the stored $\overline{W}$ to $-W$. Note that if the current input bit is zero, the multiplication result $Q[7:0]$ is always zero, so transforming $\overline{W}$ to $-W$ before multiplication is redundant. Therefore, the $+1$ compensation is only necessary when both the flip signal and current input bit equal one. To avoid compensation addition logic overhead incurred for each row, we accumulate different row's compensations in advance in the input driver and add the compensation sum to the array adder directly.

C. Technique Evaluation

Fig. 8 shows the evaluation for the FACI and input feeder. We select different MAC parallelisms ranging from 16 to 128 to evaluate our effects for hard error repair overhead reduction. MAC parallelism is the count of weight rows that are simultaneously read out and accumulated to complete MAC operations. Our MAC parallelism is 16, while other MAC parallelism results are based on simulation. The baseline is digital CIM using conventional column repair, where each read-out row has a redundant column and detour logic for repair. The implementation of FACI and input feeder is much more lightweight than the complex and expensive detour logic. Only a small number of hard errors are randomly distributed in SRAM, so storing the addresses where faults occur in the hard error recorder only consumes little storage overhead. Besides, the input feeder only manages the input inversion of the 16 current inputs to 16 activated weight rows, without the need to care for all input features. Therefore, only 16 registers and inverters need to be reserved, so the input feeder's implementation logic is simple, no matter whether the input bitwidth is 8 or 16. Compared with the baseline, our redundancy-free hard error repair achieves 7.23-to-8.47× power and 12.87-to-18.16× area savings. As more rows are simultaneously read out, the detour logic overhead grows proportionally in the baseline, and our method can save more.

To further evaluate the benefits of the designed input feeder that inverts inputs on chip, we compare the area and power overhead between the input feeder and storing both the flipped and non-flipped inputs in the input buffer. In other words, the baseline is also based on the redundancy-free hard error repair. Their differences lie in how we invert the inputs, i.e., online or offline. The baseline inverts the inputs offline and stores both the flipped and non-flipped inputs in the input buffer, which is a naive implementation, so the input feeder is not required. In contrast, ETCIM employs the input feeder to invert the input online based on the flip signal of the activated weight row and does not store the two copies of inputs in the input buffer. As the input feeder's area and power overhead are much less than those of the input buffer, ETCIM delivers 1.19× area saving and 1.17× power reduction in total compared to the baseline.

V. BW-ECC CIM Macro

A. Soft MAC Error Correction Workflow

As mentioned in Section III, the implemented CIM macro comprises 32 weight blocks. To simplify the illustration of our soft MAC error correction principle, we begin with a simple example with four weight blocks. As shown in Fig. 9(a), in each CIM row, besides weight blocks, each row comprises the checksum and residue code that are encoded offline and stored in CIM. The checksum stores the sum of all weight blocks. The residue code stores selective blocks' mod3 addition results.

Fig. 9(b) illustrates the implementation details of the selective mod3 addition and its comparison with traditional hamming code. Hamming code selectively combines data bits to form parity bits using bitwise XOR. Both data bits that need protection and parity bits are individual bits. BW-ECC also selectively combines weight blocks to form residue codes. The selective combination method is the same as hamming code. Their differences lie in that, in BW-ECC, each weight block that needs protection has 8-b precision. Therefore, we extend each 1-b parity bit in hamming code to 2-b residue code, which stores the mod3 value (0, 1, or 2) of selective blocks' addition results. The reason for the employment of the mod3 residue code is elaborated as follows: if some weight bit flips from 0 to

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

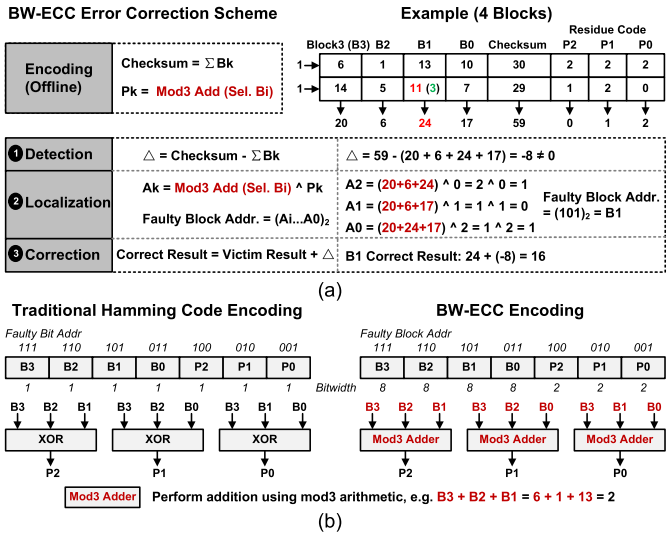


Fig. 9. Soft MAC error correction workflow. (a) BW-ECC error correction scheme and a four-block example. ❶ Error detection, ❷ faulty block localization, and ❸ error correction. (b) Comparison between traditional hamming code and our BW-ECC.

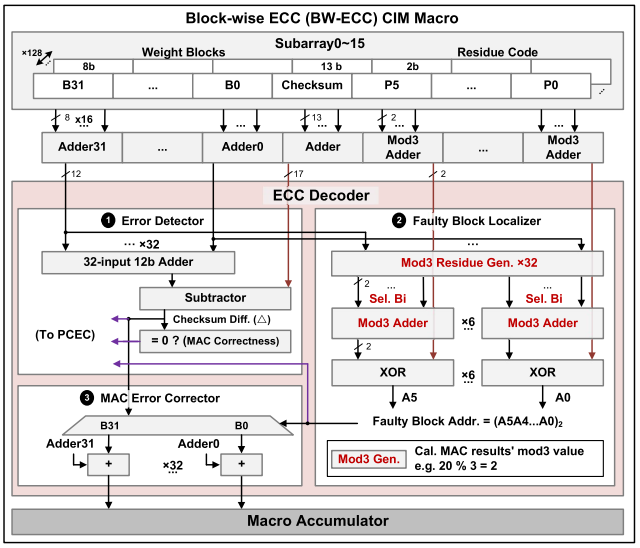


Fig. 10. BW-ECC CIM macro architecture: ❶ error detector, ❷ faulty block localizer, and ❸ MAC error corrector.

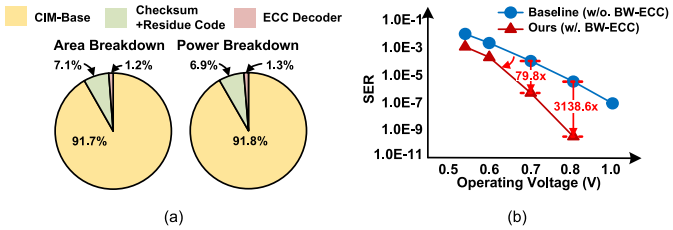


Fig. 11. Evaluation for the BW-ECC CIM on soft MAC error correction. (a) ECC code and ECC decoder overhead in a BW-ECC CIM macro. (b) SER reduction at different voltages. Baseline: conventional digital CIM without BW-ECC.

1 or from 1 to 0, this weight value will occur a $\pm 2^i$ deviation. The deviation will propagate to the MAC result after multi-row addition, so the victim MAC result's mod3 value will mismatch the correct value. Thus, by comparing MAC results' mod3 addition value with the residue code again, we can locate the faulty block address according to the comparison mismatch situation. The decoding of the faulty block address is similar to the process of faulty bit address decoding in hamming code.

During computation, each cycle's MAC errors are detected and corrected via ECC decoding. The first step is error detection. We first compare the sum of all weight blocks' MAC results with the checksum MAC results. The difference is the deviation of the victim MAC result from the correct value in the faulty block. If the checksum difference is not zero, the error is detected. In this example, the original weight in Block1's second row is 3 [$(00000011)_2$]. However, after a cell error happens, the weight is corrupted to 11 [$(00001011)_2$], so the victim MAC result is 8 greater than the original value, leading to a non-zero checksum difference.

The second step is faulty block localization. We selectively add weight blocks' MAC results using mod3 arithmetic, which is the same as encoding. If no cell error happens, weight MAC results' mod3 addition values are the same as residue code's MAC results in the mod3 domain. Otherwise, once a cell error happens in a block and propagates to this block's MAC result, its mod3 value will mismatch the correct value, which is reflected in non-zero Ak to decode the faulty block address. In the example, Block1's victim MAC result (24)'s mod3 value is 0, but the correct MAC result (16)'s mod3 value is 1. Since Block1 is involved in the calculation of residue codes P2 and P0, weight MAC results' mod3 addition values mismatch P2 and P0's MAC results. Therefore, $(A2A1A0)_2$ equals $(101)_2$, which indicates that the faulty block is Block1.

The third step is the error correction. After the faulty block and the deviation from the correct value are known, the victim MAC result adds the checksum difference to correct the MAC error. Block1's correct result is $24 + (-8) = 16$.

B. BW-ECC CIM Architecture

Next, we introduce BW-ECC CIM's detailed architecture to explain how it works to support the above three steps, as shown in Fig. 10. The ECC encoder is inserted between the array adders and the global accumulator, which protects each block's MAC correctness before MAC results are sent to the accumulator for partial sum accumulation. The three steps such as error detection, localization, and correction in ECC decoding are realized by the three modules in the ECC decoder. Since there are 32 weight blocks in the actual implementation, six residue codes (P0–P5) are required, and the selective combination method follows the (38, 32) hamming code realization. To save the overhead of weight MAC results' mod3 addition, we first generate 32 MAC result's mod3 values and then combine them using mod3 addition, avoiding the repeated transformation overhead for the original MAC results into the mod3 domain.

C. Technique Evaluation

Fig. 11 shows the evaluation for the BW-ECC CIM. Fig. 11(a) breaks down the area and power of a BW-ECC CIM macro. CIM-Base represents the base digital CIM macro logic excluding the ECC-related overhead. ECC code includes residue code and checksum's storage and MAC operation overhead. Due to the low proportion of ECC code compared to weight blocks and the lightweight error correction design, the ECC-related logic (including ECC code and ECC decoder)

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

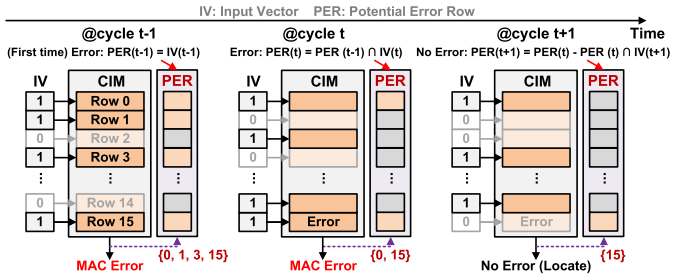


Fig. 12. Soft cell error correction example (error row = row 15).

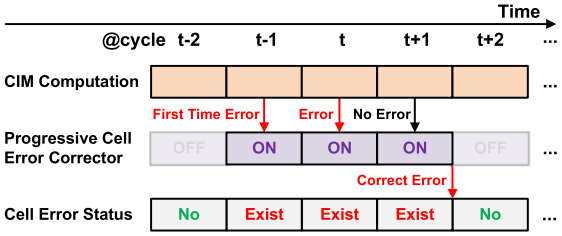


Fig. 13. Soft error correction workflow chart.

only consumes 8.3% area and 8.2% power overhead. Fig. 11(b) shows the soft error rate (SER) reduction at different voltages. The SER is the probability of an SRAM cell occurring soft errors. The baseline SER versus voltage curve is the typical results [10], [28] at the read-critical process corner (temperature: 22 °C), and our curve is simulated based on it. In comparison to digital CIM without BW-ECC, our MAC error correction achieves 79.8× and 3138.6× SER reduction at 0.7 and 0.8 V, respectively.

VI. Progressive Cell Error Corrector

A. Soft Cell Error Correction Workflow

Although BW-ECC can correct MAC errors, we still need to correct the original flipped SRAM cell that causes MAC errors to avoid uncorrected cell errors' accumulation, as discussed in Section II-D. Since MAC results merge the information of multiple read-out rows and confuse the distinction of correct and faulty SRAM rows, we need to locate the error row between multiple simultaneously activated rows. BW-ECC's MAC error correction ability enables CIM to continue computation with the presence of cell error. Meanwhile, PCEC utilizes each computation cycle's MAC correctness and different activated rows to gradually locate and correct the error row during computation.

We begin with a simple example of soft cell error correction shown in Fig. 12, with its workflow chart plotted in Fig. 13. When a cell error happens in @cycle $t-1$, CIM detects an MAC error at the first time. Therefore, the PER is all activated rows in this cycle (e.g., {0, 1, 3, 15}), as each row appearing a cell error will lead to incorrect MAC results.

Then, the PER scope is narrowed in consecutive computation cycles. If this cycle's MAC result is not correct, we can know that the error row is activated in this cycle. For instance, in @cycle t, the activated rows are Rows 0, 2, 14, and 15, and the error row is one of them. Therefore, PER can be reduced to the intersection between the last cycle's PER ({0, 1, 3, 15}) and this cycle's input vector ({0, 2, 14, 15}), so only two rows (Rows 0 and 15) may have the cell error now.

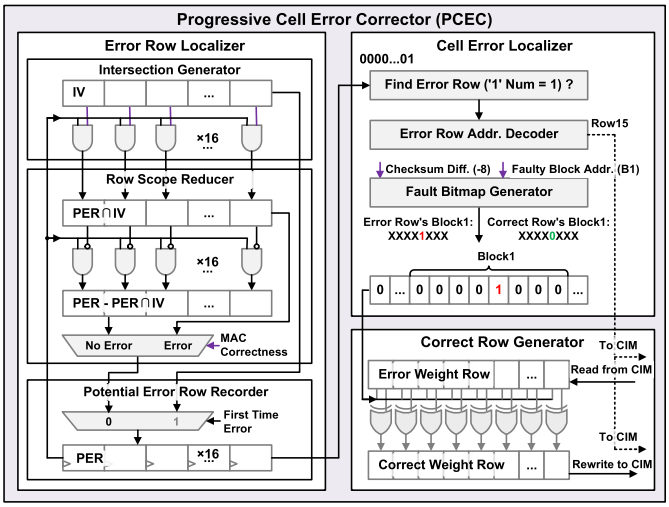


Fig. 14. PCEC architecture.

Otherwise, if this cycle's MAC result is correct, it indicates that the error row is not activated in this cycle. In @cycle $t+1$, since this cycle's activated rows (Rows 0, 3, and 14) do not have the error row, PER can exclude its intersection with the current input vector (Row 0). Once the PER has only one row, the error row (Row 15) is found.

B. PCEC Architecture

Fig. 14 demonstrates the PCEC's detailed architecture. It comprises an error row localizer, a cell error localizer, and a correct row generator. As illustrated in Section VI-B, we represent PER using bitmaps and store them in the registers. The "1" denotes rows that are possible to have errors, while "0" denotes rows that are not. The error row localizer gradually reduces the PER following the equation in Fig. 12. Note that to ensure effective PER scope reduction, we only activate PCEC when the input data contain both 1s and 0s. If the 16 subarrays' inputs are all zeros under high input sparsity, this cycle's computation is skipped for speedup. Meanwhile, PCEC is deactivated to avoid useless error row localization. Once the error row is found, the cell error localizer locates the cell error position in this error row based on the checksum difference and faulty block address from PCEC. Since each cell flip will result in a $\pm 2^i$ deviation in the victim MAC result, different error columns in the faulty block can be distinguished via their unique checksum difference. For instance, the deviation of the victim MAC result from the correct value is $+2^3$, so we can know that its root cause is the 5th bit in Block1 flipping from 0 to 1. After the cell error position is known, the error weight row is fetched from CIM according to the decoded error row address. Then, we employ XOR gates to correct the flipped SRAM cell. Finally, the correct row is generated on chip and rewritten back to CIM.

As described in Section II-C, we only consider correcting single-bit errors in a row. To avoid multi-bit errors, we track the located error row from PCEC during computation. If a row is recorded more than twice in a short period, we consider a multi-bit error occurring in this row. Then, we stall computation, reload the original correct row from off-chip, and rewrite the error row using the correct row.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

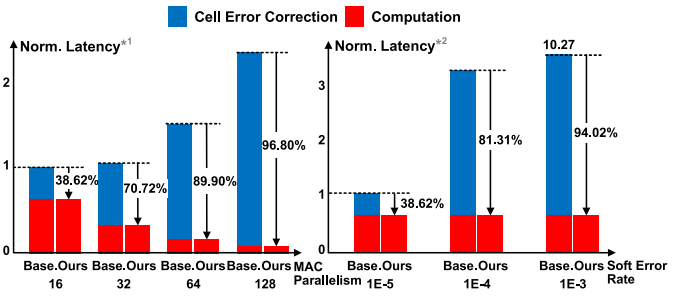
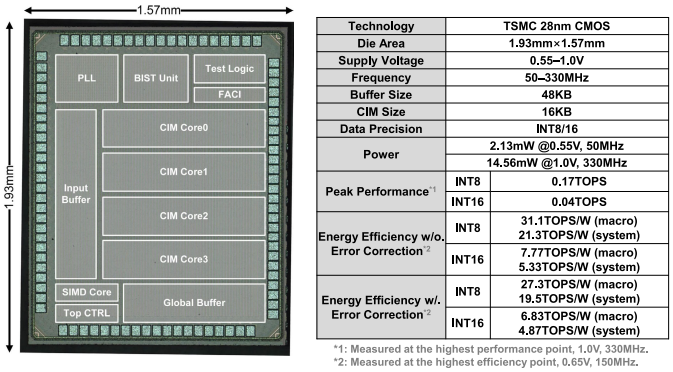


Fig. 15. Evaluation for the PCEC on hiding soft cell error correction latency, evaluated on ResNet-50 inference at INT8. Baseline: digital CIM with BW-ECC but without PCEC, which stalls computation and reads out SRAM rows in turn to locate the error row after detecting an MAC error. *1: SER is 1E-5. *2: the MAC parallelism is 16.



Technology		TSMC 28nm CMOS
Die Area		1.93mm×1.57mm
Supply Voltage		0.55–1.0V
Frequency		50–330MHz
Buffer Size		48KB
CIM Size		16KB
Data Precision		INT8/16
Power		2.13mW @0.55V, 50MHz
		14.56mW @1.0V, 330MHz
Peak Performance*1	INT8	0.17TOPS
	INT16	0.04TOPS
Energy Efficiency w/o. Error Correction*2	INT8	31.1TOPS/W (macro) 21.3TOPS/W (system)
	INT16	7.77TOPS/W (macro) 5.33TOPS/W (system)
Energy Efficiency w/. Error Correction*2	INT8	27.3TOPS/W (macro) 19.5TOPS/W (system)
	INT16	6.83TOPS/W (macro) 4.87TOPS/W (system)

*1: Measured at the highest performance point, 1.0V, 330MHz.
*2: Measured at the highest efficiency point, 0.65V, 150MHz.

Fig. 16. ETCIM's chip micrograph and specifications.

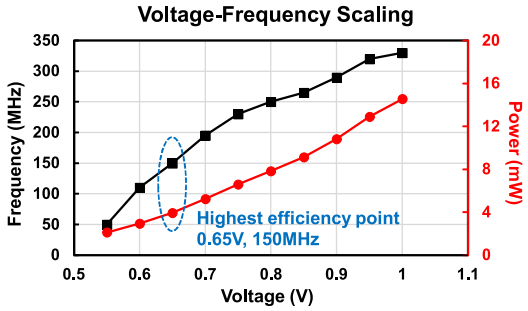


Fig. 17. ETCIM's voltage–frequency scaling curves.

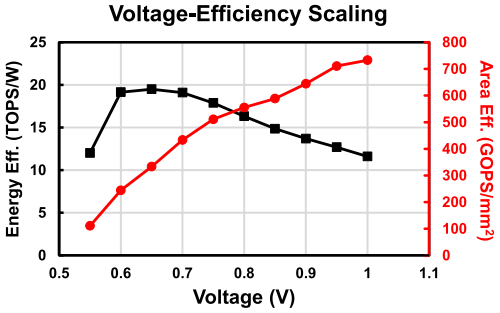


Fig. 18. ETCIM's voltage–efficiency scaling curves. The energy and area efficiency are plotted at IN8 precision.

C. Technique Evaluation

Fig. 15 shows the evaluation for the PCEC. The benchmark is ResNet-50, where weights and inputs are quantized to 8 b for all layers. We select different MAC parallelisms ranging from 16 to 128 and different SERs to evaluate PCEC's effects for hiding soft cell error correction latency. The different networks' trends are similar, so we only present a set of results on one network. The baseline is digital CIM with BW-ECC but without PCEC, which stalls computation and reads out SRAM rows in turn to locate the error row after detecting an MAC error. With the growth of MAC parallelisms, the computation latency decreases since more rows are simultaneously read out for computation. However, the baseline's cell error correction latency increases as more rows need to be read out in turn to locate the error row. Therefore, our PCEC delivers more latency reduction as the MAC parallelism increases, ranging from 38.62% to 96.80%. Similarly, our solution achieves up to 16.72× speedup with the growth of SERs, as errors occur more frequently and the baseline's row-by-row reading out latency increases. Thanks to the efficient error row localization mechanism and lightweight hardware logic, PCEC only incurs 1.4% area and 1.6% power overhead.

VII. Experimental Results

This section first describes ETCIM's measurement results from a 28-nm fabricated chip. We then provide the test platform setup for evaluation. Next, we elaborate our error injection methodology for individual technique evaluation. We conduct a comprehensive evaluation for ETCIM's error correction techniques, including benefits of accuracy and energy efficiency. Finally, we compare ETCIM with state-of-the-art works.

A. Chip Micrograph and Summary

Figs. 16–18 present the ETCIM's chip micrograph, specifications, and the scaling curves between frequency, power, energy efficiency, and area efficiency versus voltage. ETCIM is fabricated in 28-nm CMOS technology, with a 1.93 × 1.57 mm^2 die area. The chip can work at 0.55∼1.0-V supply, 50∼330 MHz with power consumption of 2.13∼14.56 mW. ETCIM can support INT8 and INT16 precision, where INT16 support is realized by concatenating two INT8 weight blocks to store INT16 weights and shift-and-adding their CIM outputs to form INT16 MAC results. The peak performance reaches 0.17 tera operations per second (TOPS) at INT8 and 0.04 TOPS at INT16 under 1.0 V, 330 MHz. The highest efficiency point is 0.65 V, 150 MHz. When the operating voltage is lower than 0.65 V, the switching speed of transistors slows down, resulting in a reduced frequency. However, at this time, the voltage approaches the threshold voltage of the transistors, leading to a subthreshold leakage current. Therefore, static power consumption becomes excessively dominant, creating an energy efficiency drop. Without error correction logic, the highest energy efficiency is 21.3 TOPS/W at INT8 and 5.33 TOPS/W at INT16 under 0.65 V, 150 MHz. With error correction logic, the highest energy efficiency is 19.5 TOPS/W at INT8 and 4.87 TOPS/W at INT16 under 0.65 V, 150 MHz. The metrics without error correction logic are evaluated on the digital CIM that excludes the three features from ETCIM. The energy efficiency gap between the digital CIM with error correction and that without can be used to evaluate the overhead of the proposed error-tolerant designs. ETCIM showcases a minimal 8.6% impact

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

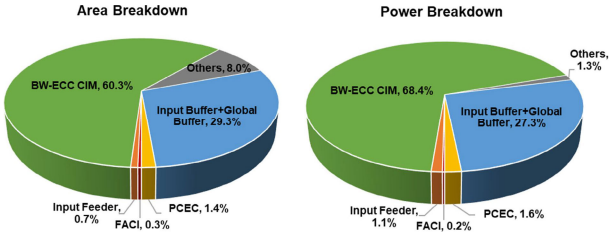


Fig. 19. ETCIM's area and power breakdown.

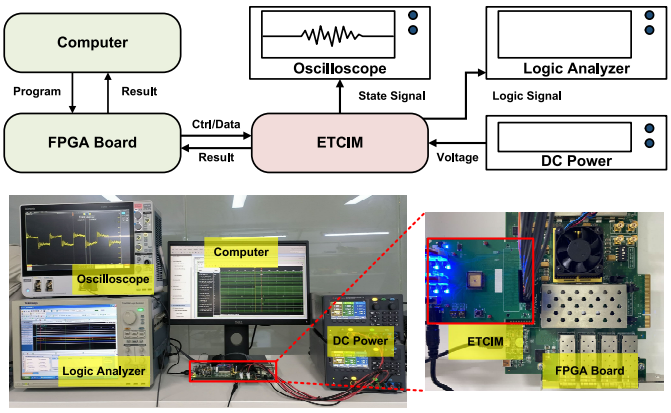


Fig. 20. Test platforms for the fabricated ETCIM processor.

on energy efficiency from error tolerance, demonstrating its practicality and feasibility for commercial designs.

We break down the area and power of ETCIM to present different techniques' overhead, as depicted in Fig. 19. FACI manages CIM cores' initialization, consuming only 0.3% in area and 0.2% in power. The input feeder is attached to each BW-ECC CIM macro for according input inversion, which consumes only 0.7% in area and 1.1% in power. PCEC corrects cell errors for each CIM macro, consuming 1.4% in area and 1.6% in power. The BW-ECC-related overhead is elaborated in Fig. 11.

B. Test Platform Setup

Fig. 20 illustrates the test platform for the fabricated ETCIM processor. The system is composed of an ETCIM test chip, Xilinx VC709 field-programmable gate array (FPGA) board, dc power, logic analyzer, oscilloscope, and host computer. The test dataset and NN model are stored in the FPGA board's DDR3 memory. The FPGA sends control signals and data to the ETCIM test chip board via the FPGA mezzanine connector (FMC). The dc power supplies 0.55∼1.0-V core voltage for the test chip. The oscilloscope and logic analyzer observe the chip's state signals and logic signals. ETCIM's computing results are collected by the FPGA and then sent to the host computer for further analysis.

C. Error Injection Methodology

To verify the effectiveness of the proposed techniques, we manually inject hard and soft errors to digital CIM. Before CIM initialization, hard errors are injected into SRAM by fixing certain SRAM cell values to 0 or 1 with the probability of hard error rate (HER), which mimics the randomly distributed

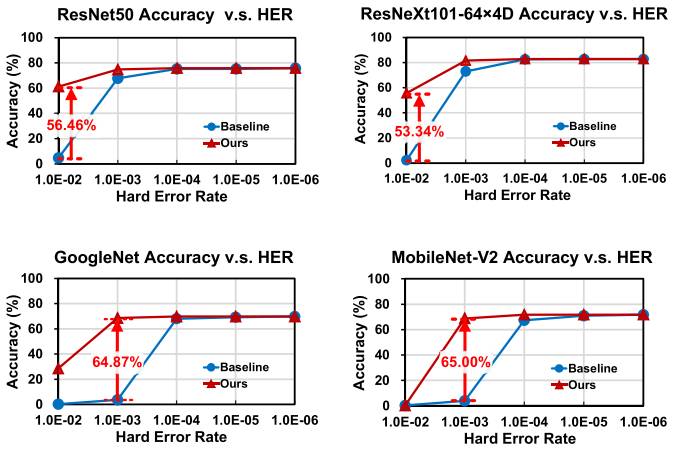


Fig. 21. Accuracy improvement from redundancy-free hard error repair on various benchmarks.

stuck-at 1 or 0 faults from chip fabrication. During each read in CIM computation, we flip cell values with a certain possibility based on the typical SER-voltage curve at 28 nm [10], [28] to inject soft errors. This mimics the read disturbance issue under low supply voltages. By manually adjusting the error injection settings, the proposed error correction techniques can be comprehensively evaluated under various experimental settings, such as different MAC parallelisms, HER and SER, and voltages.

D. Evaluation for Error Correction Techniques

1) Accuracy Improvement: To evaluate the effects of our hard error repair and soft error correction, we inject hard and soft errors separately, and compare the inference accuracy between the baseline and ETCIM. Fig. 21 presents the accuracy improvement from our redundancy-free hard error repair on various benchmarks. Weights and inputs are quantized to 8 b for all benchmarks. The baseline is the same digital CIM without hard error repair. To comprehensively evaluate our effects under different scenarios, we vary the HER of an SRAM cell from 1.0E-6 to 1.0E-2 and observe the corresponding accuracy of the baseline and ETCIM. When the HER is low, the NN can recover the inference accuracy due to its inherent error resilience. However, as the increasing HER exceeds NN's inherent resilience, the baseline suffers from significant accuracy loss. In contrast, our redundancy-free hard error repair counteracts the impacts of at least one hard error per SRAM row, largely recovering the accuracy under high HERs. The accuracy improvement reaches up to 56.46%, 53.34%, 64.87%, and 65.00%, respectively, on ResNet50, ResNeXt101-64 × 4-D, GoogleNet, and MobileNet-V2.

Fig. 22 presents the accuracy improvement from our run-time soft MAC and cell error correction on various benchmarks. Weights and inputs are also quantized to 8 b. The baseline is the same digital CIM without soft error correction. Since low voltage operation is a typical scenario that reduces SRAM cells' resilience to soft errors, we use voltage scaling to analyze how our method performs under different voltages. Compared to the baseline, we improve the inference accuracy by 32.57%, 48.39%, and 52.51%, respectively, on ResNet50, ResNeXt101-64 × 4-D, and GoogleNet, at 0.6-V operating voltage. The accuracy improvement of MobileNet-V2 is

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

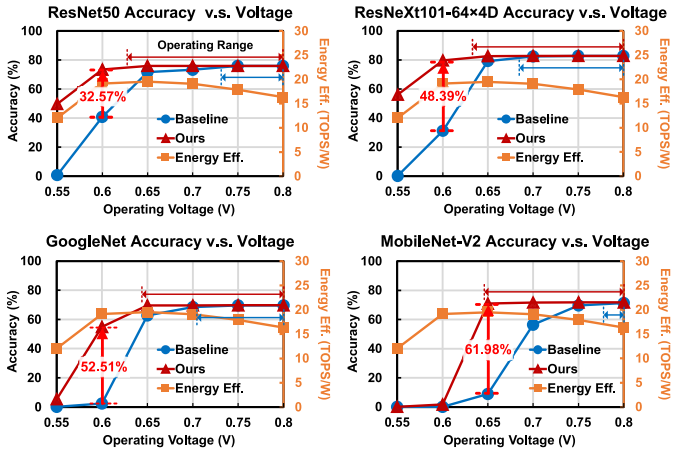


Fig. 22. Accuracy improvement from run-time soft MAC and cell error correction on various benchmarks, and operating range comparison between baseline and ours. Operating range: the effective working voltage region of digital CIM, where the inference accuracy loss is lower than 1%.

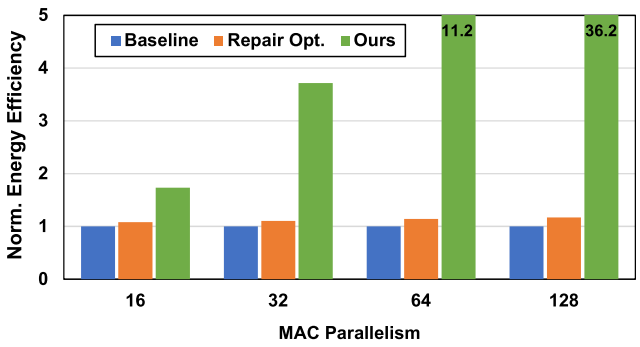


Fig. 23. Energy efficiency improvement breakdown of different techniques under various MAC parallelisms. SER is 1E-5. "Baseline": digital CIM with only redundant column repair and BW-ECC for hard and soft error corrections. "Repair Opt.": baseline + redundancy-free repair. "Ours": baseline + redundancy-free repair + run-time cell error correction.

61.98% when operating at 0.65 V. The BW-ECC corrects most of the MAC errors, and the PCEC corrects the underlying cell errors to avoid uncorrected errors' accumulation, maintaining the MAC error correction capability for subsequent errors.

Besides the effects on accuracy improvement at the same operating voltage, our soft error correction also allows digital CIM to operate at lower voltages to improve energy efficiency. To quantify its effectiveness, we plot the operating range of the baseline and ours on different benchmarks. The operating range is defined as the effective working voltage region of digital CIM where the accuracy loss is lower than 1%. Digital CIM can only work at the operating range to maintain acceptable inference accuracy. Thanks to our run-time soft MAC and cell error correction, ETCIM can maintain its functional correctness at a wider operating range. Then, we fine-tune the voltage in the operating range to find the optimum operation point that achieves the highest energy efficiency, according to the energy efficiency versus voltage curve. Since the highest efficiency point [(0.65 V, 150 MHz), see Fig. 17] is inside the operating range for all benchmarks, the four networks' optimum operation points are all at 0.65 V. In contrast, the baseline can only work at high voltages to avoid accuracy loss, so its achievable highest energy efficiency is lower than ours. It proves that our soft error correction enables edge devices to have a wider operating range to fine-tune the voltage for higher energy efficiency, which is helpful in scenarios with a tight power budget.

2) Energy Efficiency Improvement: Fig. 23 demonstrates a technique breakdown analysis to evaluate different error correction techniques' benefits on energy efficiency improvement. We select different MAC parallelisms ranging from 16 to 128. The baseline is the same digital CIM with conventional error correction techniques. Since conventional digital CIM can only employ column repair to correct hard errors and lack soft error tolerance abilities, we equip the baseline with our BW-ECC technique (Feature2a) to enable protection against both hard and soft errors, which is the same as ETCIM. Cell error correction is realized by stalling computation and reading out SRAM rows in turn. We set the SER with ECC as 1E-5. Then, we enhance the baseline with hard error repair optimization (Feature1). The eliminated huge detour logic overhead contributes to 1.08× to 1.17× energy efficiency improvements, under different MAC parallelisms. Our ETCIM further enhances the baseline with run-time cell error correction (Feature2b). The concealed cell error correction latency brings more effective computation time, which improves performance, and the entire energy efficiency improvement reaches 1.73× to 36.2×. The growth of MAC parallelism increases the detour logic overhead and row-by-row cell error correction latency for the baseline, so our optimized hard and soft error correction techniques deliver more benefits.

E. Comparison With the State-of-the-Art Works

Table I shows a detailed comparison with the state-of-the-art CIM processors. ISSCC'22-16.3 [5] is a ReRAM CIM-based processor, which integrates ECC capability to protect against MAC errors in ReRAM CIM from inherent device variation. ISSCC'23-29.5 [4] is also a ReRAM CIM-based processor, whose ReRAM macro can be configured as an error-correcting or a regular CIM macro depending upon the error compensation requirement. ISSCC'23-33.4 [3] is a MRAM CIM architecture, with column repair approach to map the address of the column with weak bit cells to an idle redundant column. ISSCC'22-15.5 [24] and ISSCC'23-7.4 [18] are the state-of-the-art digital CIM processors, without error correction abilities. As the first digital CIM processor with error tolerance capability, ETCIM features redundancy-free repair for hard errors and run-time error correction for soft errors.

To compare the efficiency loss from error correction across different CIM processors, we list their energy efficiency with and without error corrections, respectively. Original architectures of [3], [4], and [5] only employ one of the ECCs or repair. Thus, we exclude their original error-tolerant support to evaluate efficiency without error correction and include both ECC and repair to compare efficiency with error correction. Mori et al. [18] and Tu et al. [24] do not have error tolerance abilities, so we equip them with a traditional column repair approach and our BW-ECC to enable error correction, which is the same as the baseline setting in Section VII-D2. Our settings have been elaborated in Section VII-A. ETCIM's peak energy efficiency with error correction reaches 19.5 TOPS/W at INT8,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON WITH THE STATE-OF-THE-ART CIM PROCESSORS

		ISSCC'22-16.3 [5]	ISSCC'23-29.5 [4]	ISSCC'23-33.4 [3]	ISSCC'22-15.5 [24]	ISSCC'23-7.4 [18]	This Work
CIM Implementation		Analog CIM	Analog CIM	Analog CIM	Digital CIM	Digital CIM	Digital CIM
Hard Error Correction		×	×	Column Repair*1	×	×	✓Redundancy-Free Repair
Soft MAC Error Correction		Bit-wise ECC*1	Bit-wise ECC*1	×	×	×	✓Block-wise ECC
Soft Memory Cell Error Correction*2		×	×	×	×	×	✓Run-time Correction
Data Precision		INT1/4/8	INT1/4/8	INT1	INT8/16, BF16, FP32	INT8/12	INT8/16
Technology		40nm	40nm	28nm	28nm	4nm	28nm
Supply Voltage (V)		0.9	0.9	1.0	0.6-1.0	0.32-1.1	0.55-1.0
Frequency (MHz)		192	100	20	50-220	410-1490	50-330
Die Area (mm^2)		25	20.25	4.5	6.69	0.0172 (macro)	3.03
Power (mW)		131.3	200.5	23.7	12.5-69.4	9.8	2.1-14.6
Peak Performance (TOPS)	INT8	0.12	0.23	0.98 (INT1)	1.35	0.86	0.17*7
	INT16	N/A	N/A	N/A	0.34*5	0.22*5	0.04*7
Area Efficiency ($TOPS/mm^2$)	INT8	0.0048	0.011	0.22 (INT1)	1.43	49.9	0.73*7
	INT16	N/A	N/A	N/A	0.36*5	12.5*5	0.18*7
Energy Efficiency w/o. Error Correction (TOPS/W)*3	INT8	0.95	1.43	48.8 (INT1)	36.5	84.8*6	21.3*8
	INT16	N/A	N/A	N/A	9.1*5	21.2*5*6	5.33*8
Energy Efficiency w/. Error Correction (TOPS/W)*4	INT8	0.92	1.12	33.2 (INT1)	14.8	2.84*6	19.5*8
	INT16	N/A	N/A	N/A	3.7*5	0.71*5*6	4.87*8

*1: Prior analog CIM processors can rely on column repair and bit-wise ECC for hard and soft error correction, since they complete accumulations in bitlines first and read out values from bitlines in the same way as SRAM, which cannot be applied to digital CIM.
*2: Prior analog CIM processors only consider MAC error from accumulated variation, without memory cell error correction.
*3: ECC or repair are excluded in [3–5] for fair comparison.
*4: Both ECC and repair are included to compare energy efficiency with error correction.
*5: The INT16 results are projected from the reported INT8 numbers.
*6: On-chip buffers are included for fair comparison. Buffer capacity is 48KB, same as ETCIM.
*7: Measured at the highest performance point, 1.0V, 330MHz.
*8: Measured at the highest efficiency point, 0.65V, 150MHz.

which is 1.32× higher than [24] and 6.87× higher than [18]. Although Mori et al. [18] achieve the highest energy efficiency without error correction, its efficiency significantly degrades if we need to correct hard and soft errors in it. This is because Mori et al.'s [18] MAC parallelism is 144, largely increasing the detour logic overhead and row-by-row cell error correction latency, which leads to 96.7% efficiency loss. In contrast, ETCIM's efficiency loss is only 8.6%, due to its efficient error-resilience designs. The redundancy-free repair eliminates the expensive detour logic via adaptive weight flipping. The BW-ECC and PCEC conceal the cell error correction latency in subsequent computation and realize run-time MAC and cell error correction, avoiding the time-consuming computation stall that limits the performance.

Although Cai et al. [3] and Chang et al. [4], [5] have employed column repair or bitwise ECC to correct hard or soft errors in analog CIMs, their methods cannot be applied to digital CIM. Typical analog CIMs complete accumulations in bitlines first and read out values from bitlines. Similar to SRAM, they also only need to read out one row of bitline values in each cycle, unlike digital CIM that reads out multiple rows simultaneously. As a result, applying column repair is feasible for analog CIM, but incurs massive detour logic overhead for digital CIM. Meanwhile, analog CIM's ECC schemes [4], [5], [7] rely on bit-level accumulation results of each column to detect the column with variation-induced errors. Instead of merging accumulation results of different weight bits like analog CIM, digital CIM generates MAC results of multi-bit weights by digital adders directly. Therefore, there are no bit-level accumulation results of each column in digital CIM, and existing bitwise ECC for analog CIM cannot correct MAC errors in digital CIM. Besides, prior analog CIM processors only consider correcting MAC errors without memory cell error correction, as they assume that all MAC errors are from accumulated variation. In other words, the accumulation of all cells' variation in a column results in MAC errors, rather than one flipped SRAM cell causing MAC errors in digital CIM. Digital CIM still needs to correct the original SRAM cell after MAC error correction, and this cell error correction topic has never been studied before.

VIII. CONCLUSION

This work proposes the first error-tolerant CIM processor ETCIM, with comprehensive hard and soft error-tolerance designs. The redundancy-free hard error repair flips weights to match SRAM, revolutionizing the conventional approach that repairs SRAM to match weights. The run-time soft error correction suits digital CIM's structure to correct MAC errors and observes subsequent computation's information to reveal the hidden error row location. While most previous research focuses on improving digital CIM's energy and area efficiency, ETCIM explores a new direction of enhancing digital CIM's reliability and robustness, which is critical for digital CIM's real deployment in commercial products. Furthermore, with technology scaling, digital CIM is more prone to hard and soft errors due to the decreasing yield and operating voltages. The HER and SER also increase as CIM processors scaling out, which is an inevitable trend driven by the dominance of large-scale models [33]. ETCIM bridges the most crucial gap for the commercialization of digital CIM, with its significance bound to reach new heights in the future.

REFERENCES

[1] Adv. Micro Devices (AMD). (2020). *Advanced Memory Device Correction (AMDC) for Servers*. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/advanced-memory-device-correction.pdf

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.

[2] AnandTech. (2020). *Early TSMC 5nm Test Chip Yields 80%, HVM Coming in H1 2020*. [Online]. Available: https://www.anandtech.com/show/15219/early-tsmc-5nm-test-chip-yields-80-hvm-coming-in-h1-2020

[3] H. Cai et al., "33.4 a 28 nm 2Mb STT-MRAM computing-in-memory macro with a refined bit-cell and 22.4–41.5TOPS/W for AI inference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 500–502.

[4] M. Chang et al., "A 73.53TOPS/W 14.74TOPS heterogeneous RRAM in-memory and SRAM near-memory SoC for hybrid frame and event-based target tracking," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 426–428.

[5] M. Chang et al., "A 40nm 60.64TOPS/W ECC-capable compute-in-memory/digital 2.25MB/768KB RRAM/SRAM system with embedded cortex M3 microprocessor for edge recommendation systems," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[6] Y.-D. Chih et al., "16.4 an 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 252–254.

[7] B. Crafton et al., "CIM-SECDED: A 40nm 64Kb compute in-memory RRAM macro with ECC enabling reliable operation," in *Proc. IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, Nov. 2021, pp. 1–3.

[8] P. Ehlig and S. Pezzino, "Error detection in sram," Texas Instrum., Dallas, TX, USA, Tech. Rep. SPRACC0A, 2017.

[9] B. Feinberg, S. Wang, and E. Ipek, "Making memristive neural network accelerators reliable," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Aug. 2018, pp. 52–65.

[10] F. Frustaci, M. Khayatzadeh, D. Blaauw, D. Sylvester, and M. Alioto, "13.8 a 32kb SRAM for error-free and error-tolerant applications with dynamic energy-quality management in 28nm CMOS," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 244–245.

[11] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[12] H. Fujiwara et al., "34.4 a 3nm, 32.5TOPS/W, 55.0TOPS/mm^2 and 3.78Mb/mm^2 fully-digital compute-in-memory macro supporting INT12×INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 572–574.

[13] W. Gul, M. Shams, and D. Al-Khalili, "SRAM cell design challenges in modern deep sub-micron technologies: An overview," *Micromachines*, vol. 13, no. 8, p. 1332, Aug. 2022.

[14] R. W. Hamming, "Error detecting and error correcting codes," *Bell Syst. Tech. J.*, vol. 29, no. 2, pp. 147–160, Apr. 1950.

[15] J. Huynh, "The AMD Athlon XP processor with 512KB L2 cache," Adv. Micro Devices, Inc., Sunnyvale, CA, USA, Tech. Rep. 26485A, 2003.

[16] I. Koren and Z. Koren, "Defect tolerance in VLSI circuits: Techniques and yield analysis," *Proc. IEEE*, vol. 86, no. 9, pp. 1819–1838, Sep. 1998.

[17] G. Li et al., "Understanding error propagation in deep learning neural network (DNN) accelerators and applications," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Nov. 2017, pp. 1–12.

[18] H. Mori et al., "A 4nm 6163-TOPS/W/b **4790 − TOPS/mm^2/b** SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 132–134.

[19] NVIDIA Corp. *NVIDIA GPU Memory Error Management*. Accessed: Nov. 5, 2024. [Online]. Available: https://docs.nvidia.com/deploy/a100-gpu-mem-error-mgmt/index.html

[20] H. Pilo et al., "An SRAM design in 65 nm and 45 nm technology nodes featuring read and write-assist circuits to expand operating voltage," in *Proc. Symp. VLSI Circuits, Dig. Tech. Papers.*, Jun. 2006, pp. 15–16.

[21] T. Song et al., "A 10 nm FinFET 128 mb SRAM with assist adjustment system for power, performance, and area optimization," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 240–249, Jan. 2017.

[22] J.-W. Su et al., "A 28 nm 384kb 6T-SRAM computation-in-memory macro with 8b precision for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Jun. 2021, pp. 250–252.

[23] M. B. Sullivan, S. K. S. Hari, B. Zimmer, T. Tsai, and S. W. Keckler, "SwapCodes: Error codes for hardware–software cooperative GPU pipeline error detection," in *Proc. 51st Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2018, pp. 762–774.

[24] F. Tu et al., "A 28nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[25] F. Tu et al., "16.4 TensorCIM: A 28 nm 3.7nJ/Gather and 8.3TFLOPS/W FP32 digital-CIM tensor processor for MCM-CIM-based beyond-NN acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 254–256.

[26] F. Tu et al., "A 28nm 15.59μJ/Token full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 466–468.

[27] F. Tu et al., "16.1 MuITCIM: A 28nm 2.24μJ/Token attention-token-bit hybrid sparse digital CIM-based accelerator for multimodal transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 248–250.

[28] F. Frustaci, M. Khayatzadeh, D. Blaauw, D. Sylvester, and M. Alioto, "SRAM for error-tolerant applications with dynamic energy-quality management in 28 nm CMOS," *IEEE J. Solid-State Circuits*, vol. 50, no. 5, pp. 1310–1323, May 2015.

[29] F. Tu et al., "ReDCIM: Reconfigurable digital computing-in-memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan. 2023.

[30] L. Wang et al., "34.9 a Flash-SRAM-ADC-Fused plastic computing-in-memory macro for learning in neural networks in a standard 14 nm FinFET process," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 582–584.

[31] Y. Wang et al., "ETCIM: An error-tolerant digital-CIM processor with redundancy-free repair and run-time MAC and cell error correction," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2024, pp. 1–2.

[32] Y. Wang et al., "TensorCIM: Digital computing-in-memory tensor processor with multichip-module-based architecture for beyond-NN acceleration," *IEEE J. Solid-State Circuits*, vol. 60, no. 2, pp. 734–747, Feb. 2025.

[33] B. Yang, J. Chen, and F. Tu, "Towards efficient generative AI and beyond-AI computing: New trends on ISSCC 2024 machine learning accelerators," *J. Semiconductors*, vol. 45, no. 4, Apr. 2024, Art. no. 040204.

[34] K. Yoshioka, "34.5 a 818–4094TOPS/W capacitor-reconfigured CIM macro for unified acceleration of CNNs and transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 67, Feb. 2024, pp. 574–576.



Yiqi Wang (Graduate Student Member, IEEE) received the B.S. degree from the School of Integrated Circuits, Tsinghua University, Beijing, China, in 2021, where he is currently pursuing the Ph.D. degree.

His research interests include deep learning, in-memory computing, and computer architecture.



Zhen He (Graduate Student Member, IEEE) received the B.S. degree from Chongqing University, Chongqing, China, in 2022. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits, Tsinghua University, Beijing, China.

His research interests include in-memory computing, computer architecture, and fault-tolerant design.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.



Chenggang Zhao received the B.S. degree from the Department of Computer Science and Technology, Tsinghua University, Beijing, China, in 2021, where he is currently pursuing the M.S. degree with the Institute for Interdisciplinary Information Sciences.

His research interests include computer architecture, tensor compiler, and GPU kernel optimization.



Zihan Wu received the M.S. degree from the School of Integrated Circuits, Tsinghua University, Beijing, China, in 2023, where he is currently pursuing the Ph.D. degree.

His research interests include in-memory computing, computer architecture, and deep learning.



Mingyu Gao received the B.S. degree in microelectronics from Tsinghua University, Beijing, China, and the M.Sc. and Ph.D. degrees in electrical engineering from Stanford University, Stanford, CA, USA.

He is currently an Assistant Professor of computer science with the Institute for Interdisciplinary Information Sciences (IIIS), Tsinghua University. His research interests lie in the fields of computer architecture and systems, including efficient memory architectures, scalable data processing, and hardware system security.



Huiming Han received the B.S. degree from the School of Electronic and Information Engineering, Hebei University of Technology, Tianjin, China, in 2005.

He is currently a Senior Engineer with Tsinghua University, Beijing, China. His research interests include circuit design, compilation optimization, and intelligent computing chip systems.



Shaojun Wei (Fellow, IEEE) was born in Beijing, China, in 1958. He received the Ph.D. degree from the Faculte Polytechnique de Mons, Mons, Belgium, in 1991.

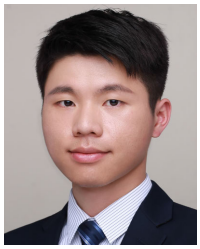
He is currently a Professor with the School of Integrated Circuits, Tsinghua University, Beijing. His main research interests include VLSI system-on-chip (SoC) design, Electronic Design Automation (EDA) methodology, and communication application-specified integrated circuit (ASIC) design.

Dr. Wei is a Senior Member of Chinese Institute of Electronics (CIE).



Yang Hu (Member, IEEE) received the B.S. degree from Tianjin University, Tianjin, China, in 2007, the M.S. degree from Tsinghua University, Beijing, China, in 2011, and the Ph.D. degree from the University of Florida, Gainesville, FL, USA, in 2017.

He was an Assistant Professor with the ECE Department, The University of Texas at Dallas, Richardson, TX, USA, from 2017 to 2021. He is currently an Associate Professor with the School of Integrated Circuits, Tsinghua University.



Fengbin Tu (Member, IEEE) received the B.S. degree from the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, China, in 2013, and the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, Beijing, in 2019.

He was a Post-Doctoral Scholar with the Scalable Energy-Efficient Architecture Laboratory (SEAL), Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbar, CA, USA, from 2019 to 2022, and a Post-Doctoral Fellow with the AI Chip Center for Emerging Smart Systems (ACCESS), Hong Kong, China, from 2022 to 2023. He is currently an Assistant Professor with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, and also the Associate Director of the Institute of Integrated Circuits and Systems. He has published two books: *Architecture Design and Memory Optimization for Neural Network Accelerators* and *Artificial Intelligence Chip Design*. His research interests include artificial intelligence (AI) chip, computing-in-memory, computer architecture, and reconfigurable computing.

Dr. Tu was a recipient of Tsinghua Excellent Dissertation Award for his Ph.D. thesis. His AI chips Thinker and ReDCIM won the 2017 ISLPED Design Contest Award and 2023 Top-10 Research Advances in China Semiconductors, respectively.



Shouyi Yin (Fellow, IEEE) received the B.S., M.S., and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2000, 2002, and 2005, respectively.

He has worked as a Research Associate with Imperial College, London, U.K. He is currently a Full Professor and the Vice Director of the School of Integrated Circuits, Tsinghua University. He has published more than 100 journal articles and more than 50 conference papers. His research interests include reconfigurable computing, artificial intelligence (AI) processors, and high-level synthesis.

Dr. Yin has served as a Technical Program Committee Member for the top VLSI and electronic design automation (EDA) conferences such as A-SSCC, MICRO, DAC, ICCAD, and ASP-DAC. He is an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS (TCAS-I), *ACM Transactions on Reconfigurable Technology and Systems (TRETS) and Integration*, and IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:19:36 UTC from IEEE Xplore. Restrictions apply.